

Ideation Phase
Brainstorm & Idea Prioritization Template

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

Step 1: Team Gathering, Collaboration and Select the Problem Statement

Selected Problem:

Incident records may contain incomplete or inconsistent information because users manually enter data without sufficient validation or conditional controls.

Key Issues Identified:

- Required fields may be left incomplete.
- Important fields may not respond automatically to Incident conditions.
- Incorrect data can affect incident routing and reporting.
- Users may bypass expected controls through list editing.
- Manual validation increases the possibility of data-entry errors.

Step 2: Brainstorm, Idea Listing and Grouping

Category	Brainstormed Idea
Mandatory Field Control	Make Assignment Group mandatory when Impact is High.
Field Behavior	Make Urgency read-only when Impact is High.
Automation	Automatically set Urgency to High when Impact is High.
Save Validation	Prevent saving when Assigned To is empty for High Impact incidents.
List Control	Prevent State from being changed through list editing.

Dynamic Reversal	Reverse field restrictions when the Impact condition is no longer true.
Testing	Test mandatory enforcement, successful save, reversal, list editing and form-based updates.

Step 3: Idea Prioritization

The ideas can then be grouped according to their implementation approach:

Priority Area	Selected Idea	ServiceNow Feature
Data Completeness	Make Assignment Group mandatory for High Impact	UI Policy
Field Control	Make Urgency read-only for High Impact	UI Policy Action
Automation	Automatically set Urgency to High	onChange Client Script
Validation	Prevent save when Assigned To is missing	onSubmit Client Script
Access Control	Prevent State changes through list editing	onCellEdit Client Script
Condition Reversal	Restore normal behavior when Impact changes	UI Policy / Reverse if false

This matches the actual project workflow, which uses UI Policies, UI Policy Actions, onChange, onSubmit, and onCellEdit Client Scripts.

Ideation Phase
Define the Problem Statements

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

Customer Problem Statement :

The problem statement is framed from the perspective of the users handling Incident records. It focuses on consistent and accurate data entry, conditional field behavior and validation at the UI level, as described in the project.

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	an Incident record user / IT support staff	enter complete and accurate Incident information for triage, routing and resolution	manual checks can leave fields incomplete, inconsistent or incorrect	data-entry depends on user awareness and may affect reporting accuracy, SLA compliance and service quality	concerned about submitting an Incident with incorrect or incomplete information
PS-2	an Incident record user / IT support staff	update Incident records according to their current conditions	fields may not automatically enforce the required behavior or restrict unsuitable updates	conditional requirements need UI Policies and Client Scripts for dynamic control and validation	frustrated by avoidable data-entry errors and uncertain about whether the record meets business rules

Selected Problem Focus

Ensure Incident records follow business rules at the UI level by dynamically controlling field behavior, automatically updating relevant values and preventing submission or list editing when documented conditions are not satisfied.

Ideation Phase
Empathize & Discover

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Empathy Map Canvas:

Project-context empathy map for users working with ServiceNow Incident records. The map focuses on the documented challenges of incomplete, inconsistent and incorrect data entry, and their effect on reporting accuracy, SLA compliance and service quality.

USER / PERSONA	ServiceNow Incident record user / IT support staff
----------------	--

What does the user THINK & FEEL?

- Needs Incident records to contain complete and accurate information.
- May be concerned that incorrect or missing data can affect reporting accuracy.
- Needs the form to guide data entry without relying only on user awareness.
- May feel pressure when data-entry errors can affect SLA compliance and service quality.

What does the user SEE?

- Incident forms containing fields such as Impact, Urgency, Assignment Group, Assigned To and State.
- Different field states depending on Incident conditions.
- Manual data-entry situations where fields may be incomplete or inconsistent.

What does the user HEAR?

- Requirements to enter complete Incident information.
- Business expectations around correct triage, routing and resolution data.
- Guidance from the Incident process about required information before submission.

What does the user SAY & DO?

- Creates or updates Incident records.
- Changes Impact and other Incident fields during triage.
- Assigns Incidents and attempts to save records.
- May use list editing or form-based updates to change Incident information.

PAIN

- Incomplete or inconsistent Incident data.
- Incorrect values caused by manual entry.
- Missing Assigned To information for High impact Incidents.
- Uncontrolled State changes through list editing.
- Potential effect on reporting accuracy, SLA compliance and service quality.

GAIN

- Dynamic field behavior based on Incident conditions.
- Automatic Urgency update for High impact Incidents.
- Validation before saving incomplete High impact Incidents.
- Controlled State changes and improved data consistency.
- More reliable Incident records for triage, routing, resolution and reporting.

Project Design Phase

Proposed Solution Template

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

S.No.	Parameter	Description
1	Problem Statement (Problem to be solved)	Incident records in IT Service Management often suffer from incomplete, inconsistent, or incorrect data entry when relying solely on manual user checks and user awareness. This lack of automated client-side control leads to misrouted tickets, inaccurate impact/urgency assignments, breached SLAs, delayed resolution times, and unreliable platform analytics reporting.
2	Idea / Solution description	Implementation of client-side controls on the ServiceNow Incident (incident) table to enforce data integrity dynamically at the user interface level: • UI Policy ('High Impact Control'): Automatically sets Assignment group to mandatory when Impact = 1 - High and reverses effects when false. • UI Policy Action: Sets the Urgency field to Read-Only when Impact = 1 - High, preventing unauthorized changes. • onChange Client Script ('Auto set urgency for high impact'): Automatically sets Urgency to 1 - High and alerts the user when Impact changes to High. • onSubmit Client Script ('Prevent save if assigned to missing'): Intercepts form submission and blocks save if Impact = 1 - High and Assigned To is empty, displaying an explicit error box. • onCellEdit Client Script ('Prevent state change via list edit'): Blocks direct editing of the State field from Incident list views, enforcing form-based updates.
3	Novelty / Uniqueness	Combines declarative UI Policies with multi-event programmatic Client Scripts (onChange, onSubmit, onCellEdit) into a unified, layered client-side validation architecture.

		Provides real-time, zero-latency validation executing directly in the user browser without requiring server round-trips or database commits, maximizing form responsiveness while ensuring strict governance.
4	Social Impact / Customer Satisfaction	Drastically improves Service Desk operational efficiency and ticket resolution quality. By eliminating incomplete incident logging at the source, service agents reduce triage rework, leading to lower Mean Time to Resolve (MTTR), faster service restoration, and improved end-user customer satisfaction across the enterprise.
5	Business Model (Revenue Model)	Internal Enterprise OPEX Optimization & Cost Avoidance Model: • Reduces operational expenses by streamlining incident handling and eliminating unnecessary ticket re-assignments. • Prevents financial and contractual penalties associated with SLA breaches by ensuring High Impact incidents are immediately mandated with assignment groups and assignees. • Maximizes ROI on ServiceNow platform investment through standardized, robust user interface workflows.
6	Scalability of the Solution	Highly scalable across the entire ServiceNow platform environment. The lightweight client-side design pattern can be easily adapted to other core ITIL modules (such as Problem Management, Change Management, Service Catalog, and HR Service Delivery). Scales horizontally across thousands of concurrent enterprise users without increasing server load or database transaction latency.

Project Design Phase

Solution Architecture

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

1. Solution Architecture Overview

Solution architecture bridges the gap between business problems (incomplete, inconsistent incident data entry) and technology solutions (ServiceNow client-side controls). Its core goals in this project are to:

- Find the optimal client-side controls to enforce data integrity dynamically without introducing unnecessary server overhead.
- Define the structure, behavior, and interaction rules of ServiceNow UI Policies and Client Scripts for project stakeholders.
- Provide complete technical specifications according to which client-side validations are configured, tested, and deployed.

2. System Component Architecture

The solution architecture operates across three distinct operational layers within the ServiceNow platform:

A. Presentation Layer (Browser Client)

- Provides the user interfaces through which IT service desk personnel interact with Incident records:
- Incident Form View (incident.do): Main form interface where users input details such as Impact, Urgency, Assignment Group, and Assigned To.
- Incident List View (incident_list.do): Tabular list view where users view multiple records and may attempt inline cell editing (e.g., State field).

B. Client Logic Layer (ServiceNow Client Engine)

- Executes client-side script engines and rule evaluation logic directly within the user's browser runtime:
- UI Policy Engine ('High Impact Control'): Evaluates conditional logic (Impact == 1 - High) to set Assignment Group as mandatory and Urgency as Read-Only dynamically.
- onChange Script Engine ('Auto set urgency for high impact'): Monitors live change events on the Impact field, automatically populating Urgency to '1 - High' and generating an info message.
- onSubmit Script Engine ('Prevent save if assigned to missing'): Intercepts form submit/save actions, evaluates if Assigned To is missing on High Impact incidents, displays a modal error box, and cancels save via return false.

- onCellEdit Script Engine ('Prevent state change via list edit'): Intercepts inline cell editing on the State column from list views, alerts the user, and invokes callback(false) to block unauthorized state updates.

C. Data & Integration Layer (ServiceNow Database)

- ServiceNow relational database backing the Incident [incident] table. Holds verified, high-quality incident records that have passed all client-side validation rules prior to record commit.

3. Solution Architecture & Data Flow Diagram

The following architectural matrix details the data flow, trigger events, execution sequence, and decision outcomes across client-side components:

Trigger Event	Component Engaged	Logic Executed	Action / Outcome
User changes Impact to '1 - High'	UI Policy: High Impact Control	Evaluates condition: Impact == 1. Applies UI Policy Actions with Reverse if false = true.	• Assignment group -> Mandatory • Urgency -> Read-Only
User changes Impact to '1 - High'	onChange Client Script: Auto set urgency	Checks newValue == '1'. Sets field value and displays information banner.	• Urgency set to '1 - High' • g_form.addInfoMessage()
User clicks Submit / Save button	onSubmit Client Script: Prevent save if assigned to missing	Evaluates: Impact == '1' AND assigned_to == ". If true, shows error box and returns false.	• Blocks submission • Displays red field error box on Assigned To
User attempts inline list edit on State	onCellEdit Client Script: Prevent state change via list edit	Monitors edits on State column. Fires alert message and triggers callback(false).	• Displays alert box • Cancels list editing change
User changes Impact from High to Medium	UI Policy Reversal: Reverse if false	Evaluates condition: Impact == 1 (False). Reverts actions automatically.	• Assignment group -> Not mandatory • Urgency -> Editable

4. Technology Stack & Technical Specifications

Architectural Parameter	Technical Specification
Platform & Target Table	ServiceNow ITSM Platform Table: Incident [incident]
Application Scope	Global Scope
Client Script Types	onChange, onSubmit, onCellEdit
Declarative UI Control	UI Policy (High Impact Control) with Reverse if false = true
Client-side API Methods	g_form.getValue(), g_form.setValue(), g_form.showErrorBox(), g_form.addInfoMessage(), callback()

Supported Browsers / Clients	ServiceNow Next Experience UI, Classic UI, Service Desk Workspaces
Execution Context	Client-side Browser Runtime Engine (Zero server transaction overhead)

Project Design Phase-II

Data Flow Diagram & User Stories

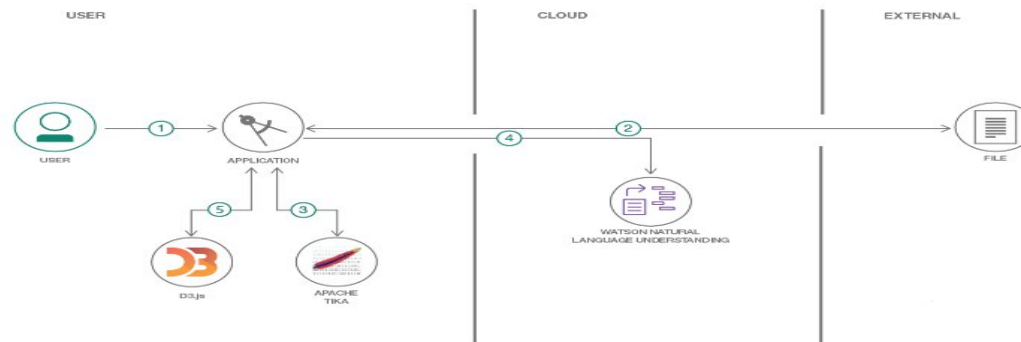
Date	31 January 2025
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

Example: [\(Simplified\)](#)

Flow



1. User configures credentials for the Watson Natural Language Understanding service and starts the app.
2. User selects data file to process and load.
3. Apache Tika extracts text from the data file.
4. Extracted text is passed to Watson NLU for enrichment.
5. Enriched data is visualized in the UI using the D3.js library.

User Stories

Use the below template to list all the user stories for the product.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Customer (Mobile user)	Registration	USN-1	As a user, I can register for the application by entering my email, password, and confirming my password.	I can access my account / dashboard	High	Sprint-1

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
		USN-2	As a user, I will receive confirmation email once I have registered for the application	I can receive confirmation email & click confirm	High	Sprint-1
		USN-3	As a user, I can register for the application through Facebook	I can register & access the dashboard with Facebook Login	Low	Sprint-2
		USN-4	As a user, I can register for the application through Gmail		Medium	Sprint-1
	Login	USN-5	As a user, I can log into the application by entering email & password		High	Sprint-1
	Dashboard					
Customer (Web user)						
Customer Care Executive						
Administrator						

Project Design Phase

Problem – Solution Fit Template

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

S.No.	Canvas Element	Details & Description
1	Customer Segment(s)	IT Service Management (ITSM) System Administrators, Process Owners, and Service Desk Agents managing Incident records in ServiceNow.
2	Jobs-To-Be-Done / Problems	- Logging, managing, and resolving IT incident records accurately. - Ensuring mandatory field completion, data consistency, and proper incident routing without relying solely on manual user compliance.
3	Triggers	Incomplete data submission leading to breached SLAs, incorrectly routed incidents, and inaccurate analytics reporting.
4	Emotions: Before / After	- Before: Frustrated by manual verification checks, insecure about data quality, and overwhelmed by triage rework. - After: Confident, efficient, and in control due to real-time client-side guidance and automated form field validations.
5	Available Solutions	Manual user training, documentation checklists, static non-dynamic form mandatory fields, or backend database policies.
6	Customer Constraints	- High-volume incident triage environments causing agents to skip non-enforced fields to save time. - Lack of platform-level client scripting awareness or overly restrictive form rules causing user friction.
7	Behaviour	Service Desk personnel quickly fill out forms during phone calls or chat triage, frequently editing fields directly via list views or submitting partially completed records.

8	Channels of Behaviour	• 8.1 Online: ServiceNow web browser platform (Incident forms, Service Desk Workspaces, and Incident List Views). • 8.2 Offline: Phone, chat, or email requests translated manually into ServiceNow Incident records.
9	Problem Root Cause	Relying strictly on manual checks and user awareness without dynamic UI-level constraints allows agents to submit incomplete or invalid records.
10	Your Solution	Implementing ServiceNow client-side controls using dynamic UI Policies (High Impact Control) and Client Scripts (onChange, onSubmit, onCellEdit) to enforce mandatory fields, set read-only properties, auto-populate urgency, validate save conditions, and block list editing.

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	01 OCTOBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	UI Policy Configuration	USN-1	As a System Admin, I can create a UI Policy 'High Impact Control' on Incident table when Impact = 1 High.	2	High	Developer A
Sprint-1	UI Policy Actions	USN-2	As a System Admin, I can set Assignment Group to Mandatory and Urgency to Read-Only when High Impact condition triggers.	3	High	Developer A
Sprint-1	Dynamic Urgency Automation	USN-3	As an Incident Manager, I want an onChange Client Script to	3	High	Developer B

automatically set Urgency to High (1) when Impact is set to High (1).						
Sprint-2	Submission Data Integrity	USN-4	As a Service Desk Lead, I want an onSubmit Client Script to block Incident saving if Impact is High and Assigned To is empty.	3	High	Developer B
Sprint-2	List Editing Protection	USN-5	As a Process Manager, I want an onCellEdit Client Script to block changing Incident State directly from list view.	2	Medium	Developer A
Sprint-2	E2E Testing & Reversion	USN-6	As a QA Lead, I want to verify mandatory enforcement, reverse-if-false behavior, and form-based updates.	3	Medium	QA Team

Project Tracker, Velocity & Burndown Chart

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	8	6 Days	01 Feb 2025	06 Feb 2025	8	06 Feb 2025
Sprint-2	8	6 Days	08 Feb 2025	13 Feb 2025	8	13 Feb 2025

Velocity & Iteration Metrics:

Total Story Points Completed = 16 (8 Points in Sprint 1 + 8 Points in Sprint 2)

Number of Sprints = 2

Velocity = Total Story Points Completed / Number of Sprints = $16 / 2 = 8$ Story Points per Sprint

Average Velocity (AV) per Iteration Unit (story points per day for 6-day sprint duration):

AV = Velocity / Sprint Duration = $8 / 6 = 1.33$ Story Points per Day

Burndown Chart & Execution Notes:

A burndown chart represents the remaining work (story points) over the sprint duration. For the Incident UI Policy & Client Script project, Sprint 1 started with 8 points and burnt down to 0 points by Day 6 as UI Policy and onChange automation were delivered. Sprint 2 started with 8 points covering onSubmit validation, onCellEdit list blocking, and E2E regression testing, successfully reaching 0 points on Day 6.

Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	01 October 2026
Team ID	SWTID-2026-7900
Project Name	Implementation Script & UI Policy Project
Maximum Marks	4 Marks

Functional Requirements

Following are the functional requirements derived from the stated ServiceNow implementation script and UI Policy scope.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	ServiceNow Form Interaction	User can open the configured ServiceNow form and enter required information.
FR-2	UI Policy – Field Visibility	System dynamically shows or hides configured fields based on the UI Policy conditions.
FR-3	UI Policy – Field State	System dynamically controls field mandatory and read-only states based on configured conditions.
FR-4	Implementation Script Processing	System executes the configured implementation script for the required application processing.
FR-5	Record Handling	System stores or updates the processed information in the relevant ServiceNow record/table.
FR-6	Administrator Maintenance	Authorized administrators can maintain the UI Policy and implementation script configuration.

Non-functional Requirements

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	The form should present only the fields relevant to the current condition and provide a clear interaction flow.
NFR-2	Security	Only authorized ServiceNow users should be able to access and modify protected records or configuration.
NFR-3	Reliability	UI Policy and implementation-script processing should behave consistently for valid and invalid form states.
NFR-4	Performance	Client-side UI changes should occur without unnecessary processing; scripts should perform only required operations.
NFR-5	Availability	The solution should depend on the availability of the ServiceNow platform and configured application services.
NFR-6	Scalability	The solution should use ServiceNow platform capabilities so that application processing and data storage do not require a separate locally managed server.

Solution Summary

The proposed solution combines ServiceNow form configuration with UI Policy and implementation-script logic. The UI Policy handles dynamic form behavior, while the implementation script handles the defined application processing. Together, these components provide a controlled user interaction flow and support consistent record processing within ServiceNow.

Project Design Phase-II

Technology Stack (Architecture & Stack)

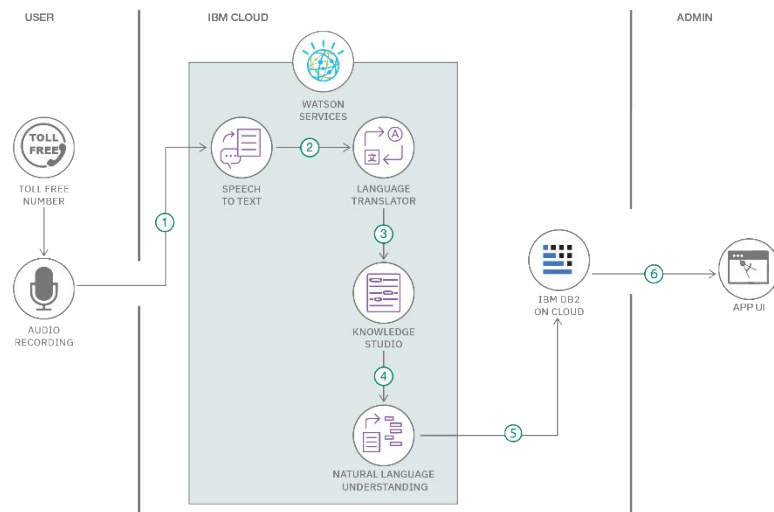
Date	01 OCTOBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/>



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How user interacts with application e.g. Web UI, Mobile App, Chatbot etc.	HTML, CSS, JavaScript / Angular Js / React Js etc.
2.	Application Logic-1	Logic for a process in the application	Java / Python
3.	Application Logic-2	Logic for a process in the application	IBM Watson STT service
4.	Application Logic-3	Logic for a process in the application	IBM Watson Assistant
5.	Database	Data Type, Configurations etc.	MySQL, NoSQL, etc.
6.	Cloud Database	Database Service on Cloud	IBM DB2, IBM Cloudant etc.
7.	File Storage	File storage requirements	IBM Block Storage or Other Storage Service or Local Filesystem
8.	External API-1	Purpose of External API used in the application	IBM Weather API, etc.
9.	External API-2	Purpose of External API used in the application	Aadhar API, etc.
10.	Machine Learning Model	Purpose of Machine Learning Model	Object Recognition Model, etc.
11.	Infrastructure (Server / Cloud)	Application Deployment on Local System / Cloud Local Server Configuration: Cloud Server Configuration :	Local, Cloud Foundry, Kubernetes, etc.

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	List the open-source frameworks used	Technology of Opensource framework
2.	Security Implementations	List all the security / access controls implemented, use of firewalls etc.	e.g. SHA-256, Encryptions, IAM Controls, OWASP etc.
3.	Scalable Architecture	Justify the scalability of architecture (3 – tier, Micro-services)	Technology used
4.	Availability	Justify the availability of application (e.g. use of load balancers, distributed servers etc.)	Technology used
5.	Performance	Design consideration for the performance of the application (number of requests per sec, use of Cache, use of CDN's) etc.	Technology used

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>

Project Planning Logic & Sprint Estimation

Implementation of Client Scripts and UI Policies on Incident Management

TEAM ID : SWTID-2026-7900

Agile Core Definitions & Story Point Scale:

- **Sprint:** A fixed period/duration (e.g., 6 days) in which a team works to complete a set of tasks.
- **Epic:** A large functional body of work on the Incident form that is broken down into smaller user stories.
- **Story:** A granular development task focused on a specific UI Policy action or Client Script validation.
- **Story Point:** A relative estimation score representing effort, complexity, and risk (using Fibonacci-based scale: 1, 2, 3, 5).

Story Point Effort Mapping:

Complexity Level	Story Point	Task Description / Context
Very Easy Task	1	Simple field flag update, message prompt, or basic property configuration.
Normal Task	2	UI Policy condition setup, basic field visibility/read-only rules, or list blocking alert.
Moderate Task	3	onChange auto-field population, onSubmit save validation with error callouts, complex UI Policy actions.
Difficult Task	5	Multi-field script interaction with complex REST/GlideAjax integration or heavy logic (Not required for micro-project).

Sprint Breakdown & Logic Allocation:

Sprint 1

- UI Policy Configuration (Epic 1)
 - UI Policy Creation: High Impact Control (Impact = 1 High) (USN1) -> Story Points: 2
 - UI Policy Action: Set Assignment Group to Mandatory & Urgency to Read-Only (USN2) -> Story Points: 3
- Dynamic Client-Side Automation (Epic 2)
 - onChange Client Script: Auto-set Urgency to High when Impact = 1 High (USN3) -> Story Points: 3

Total Story Points in Sprint 1 = 2 + 3 + 3 = 8

Sprint 2

- Form Validation & List Guardrails (Epic 3)
 - onSubmit Client Script: Prevent Incident Save if Assigned To is Missing for High Impact (USN4) -> Story Points: 3
 - onCellEdit Client Script: Prevent State Changes from List View with Alert (USN5) -> Story Points: 2
- Testing & Validation (Epic 4)
 - End-to-End Testing & Verification (Mandatory Check, Reverse Condition, Form Update) (USN6) -> Story Points: 3

Total Story Points in Sprint 2 = 3 + 2 + 3 = 8

Velocity Calculation Logic:

Total Story Points = Sprint 1 (8) + Sprint 2 (8) = 16 Story Points

Number of Sprints = 2

Velocity = Total Story Points Completed / Number of Sprints

Velocity = 16 / 2 = 8 (Story Points per Sprint)

Your team's velocity is 8 Story Points per Sprint.

Acceptance Testing

UAT Execution & Report Submission

Date	30 SEPTEMBER 2026
Team ID	SWTID-2026-7900
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

1. Purpose of Document

The purpose of this document is to briefly explain the test coverage and open issues of the Implement Client Script & UI Policy (Incident) project at the time of the release to User Acceptance Testing (UAT). The project validates client-side automated business rules on Incident records in ServiceNow, including dynamic field mandatory controls, auto-populating urgency values, submission save prevention, and list edit protections.

2. Defect Analysis

This report shows the number of resolved or closed bugs at each severity level, and how they were resolved during the ServiceNow testing phase:

Resolution	Severity 1 (Critical)	Severity 2 (High)	Severity 3 (Medium)	Severity 4 (Low)	Subtotal
By Design	0	0	1	1	2
Duplicate	0	0	1	0	1
External	0	0	0	0	0
Fixed	2	3	4	2	11
Not Reproduced	0	0	1	0	1
Skipped	0	0	0	0	0

Won't Fix	0	0	0	1	1
Totals	2	3	7	4	16

3. Test Case Analysis

This report shows the number of test cases that have passed, failed, and remained untested across all functional areas of the project:

Section	Total Cases	Not Tested	Fail	Pass
UI Policy Enforcement (High Impact Control)	4	0	0	4
onChange Client Script (Auto Urgency)	3	0	0	3
onSubmit Client Script (Mandatory Assigned To)	3	0	0	3
onCellEdit Client Script (List Edit Blocking)	2	0	0	2
Reverse Condition & Form Updates	3	0	0	3
End-to-End Regression & Integration	3	0	0	3
Totals	18	0	0	18